Module 3 — Class 3 Assignment: Categorical Encoding

Type-it-yourself exercise

UN AI/ML Mentorship

Module 3 — Class 3 Assignment: Categorical Encoding

Topic: Encoding — get_dummies, OneHotEncoder, binary mapping

Goal: Open a **fresh Google Colab notebook** and type the code yourself by following the steps below. Do NOT copy-paste from the working notebook. Typing it yourself is how you learn the syntax.

How to submit

1. Open Google Colab → File → New Notebook.
2. Type the code from each step below. Add a short `#` comment on EVERY cell so we know you understand it.
3. Save the notebook as `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`.
4. Push it to your team repo at `module-<X>/class_<Y>/submissions/<YourName>/`.

Step 0 — Load the data

```
import pandas as pd
url = 'https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Cu
df = pd.read_csv(url)
```

Step 1 — List all object (text) columns

What to do: Use select_dtypes to filter.

Type this in a new cell:

```
obj_cols = df.select_dtypes(include='object').columns.tolist()
print('Total object columns:', len(obj_cols))
print(obj_cols)
```

Expected output:

```
Total: 17 columns, list shown.
```

Step 2 — value_counts on gender

What to do: See how the values are distributed.

Type this in a new cell:

```
df['gender'].value_counts()
```

Expected output:

```
Male      3555\nFemale    3488
```

Step 3 — One-hot encode InternetService with pd.get_dummies

What to do: Make sure to add a prefix.

Type this in a new cell:

```
dummies = pd.get_dummies(df['InternetService'], prefix='Internet')
print('Shape:', dummies.shape)
dummies.head()
```

Expected output:

```
Shape: (7043, 3) - Internet_DSL, Internet_Fiber optic, Internet_No
```

Step 4 — Same column but with drop_first=True

What to do: Show that one column is removed.

Type this in a new cell:

```
dummies2 = pd.get_dummies(df['InternetService'], prefix='Internet', drop_first=True)
print('With drop_first:', dummies2.shape)
dummies2.head()
```

Expected output:

```
Shape: (7043, 2)
```

Step 5 — OneHotEncoder on TechSupport

What to do: Use sklearn's OneHotEncoder.

Type this in a new cell:

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False)
encoded = ohe.fit_transform(df[['TechSupport']])
encoded_df = pd.DataFrame(encoded, columns=ohe.get_feature_names_out(['TechSupport']))
encoded_df.head()
```

Expected output:

```
3-column DataFrame: TechSupport_No, TechSupport_No internet service, TechSupport_Yes
```

Step 6 — Binary Yes/No mapping

What to do: Use map with a dictionary.

Type this in a new cell:

```
df['Partner_bin'] = df['Partner'].map({'No': 0, 'Yes': 1})
df[['Partner', 'Partner_bin']].head()
```

Expected output:

```
First few rows showing Yes -> 1, No -> 0
```

Step 7 — Encode many columns at once

What to do: Pass a list of columns to get_dummies.

Type this in a new cell:

```
multi = pd.get_dummies(df[['gender', 'InternetService', 'TechSupport', 'Contract']], drop_first
print('Shape:', multi.shape)
multi.head()
```

Expected output:

```
Shape: (7043, ~7-8 columns)
```

Checklist before you submit

- ☐ All cells run without error
- ☐ Every code cell has a # comment in your own words
- ☐ Outputs are visible (you saved AFTER running)
- ☐ File name is correct: `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`

Deadline: before next class.